HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

Logic Design

(CO3097)

Report Lab 1

Ring Flasher Design and Simulation

Advisor:	Tôn Huỳnh Long	
Student:	Nguyễn Chí Thanh	2313078
	Nguyễn Việt Hoàng	2311066
	Lê Đức Tài	2312995
	Mai Xuân Nhựt	2312549

Ho Chi Minh City, April 12, 2026

Mục lục

1 Interface **4**

2 Functional implementation **6**

2.1 PWM Generator Module 6

2.2 Clock Divider Module 6

2.3 Controller Module 6

2.4 Ring Flasher Module 7

3 Internal implementation **8**

3.1 Overall architecture 8

3.2 State machine 12

4 Simulation **14**

4.1 Testbench 14

4.2 Simulation results 15

1 Interface

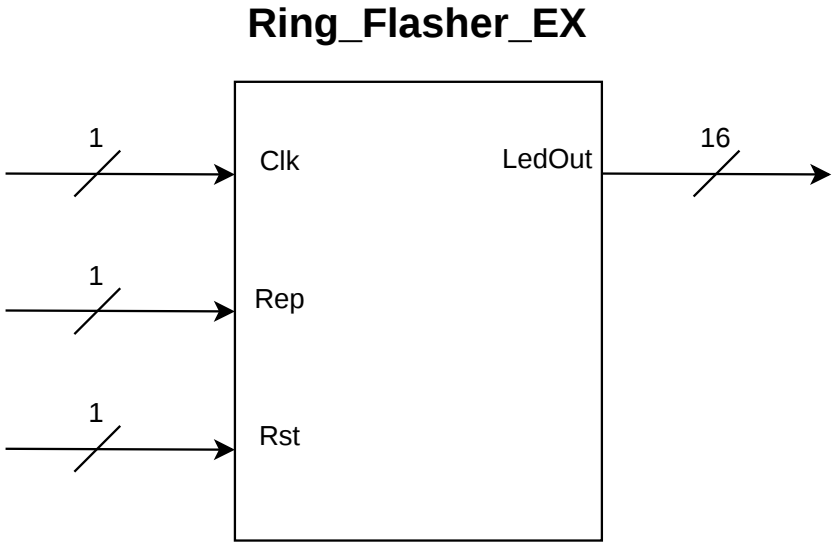

Image 1.1: The interface of the Ring Flasher module.

Bảng 1.1: Interface of Ring_Flasher_EX

Signal	**In/Out**	**Width**	**Description**
Clk	Input	1	System clock signal
Rep	Input	1	Control signal to repeat the LED pattern
Rst	Input	1	Reset signal (active high)
LedOut	Output	16	Output signal driving 16 LEDs

2 Functional implementation

2.1 PWM Generator Module

The `PWM` module is responsible for controlling the brightness of individual LEDs by modulating the output duty cycle based on a 3-bit input level.

- **Period Management**: It utilizes a 3-bit counter, `pwm_cnt`, which cycles from 0 to 4 to establish the pulse period.
- **Duty Cycle Logic**: The `ledout` signal is driven high whenever the current counter value is less than the specified `level`.
- **Error Handling**: The module includes logic to force a 100% duty cycle (`ledout = 1`) if the input level exceeds the maximum value of 5, preventing output glitche.

2.2 Clock Divider Module

The `clock_divider_module` scales the high-frequency system clock down to a timing interval suitable for human-perceivable LED transitions.

- **Dynamic Counting**: It calculates a `MAX_COUNT` value using parameters for clock frequency (`CLK_FREQ_HZ`) and desired step time (`STEP_TIME_S`).
- **Enable Control**: The timing logic is gated by an `enable_timer` signal, ensuring the sequence only advances when active.
- **Tick Generation**: Upon reaching the calculated count, it asserts a single-cycle `step_tick` (referred to as `divclk` in the top module) to trigger state updates.

2.3 Controller Module

The `Control` module implements a Finite State Machine (FSM) to manage the movement patterns of the LED ring.

- **State Machine**: The logic transitions through four states: `IDLE_STATE`, `CW_STATE` (Clockwise), `ACW_STATE` (Anti-clockwise), and `OFF_STATE`.
- **Position Calculation**: It determines the active LED index (`cur`) by incrementing or decrementing the position register based on the current state and the `rep` (repeat) input.
- **Gating Logic**: It manages the `ena_div` signal to enable the system clock divider and monitors the `off_all` feedback to determine when to return to the idle state.

2.4 Ring Flasher Module

The `Ring_Flasher_EX` is the top-level module that integrates the controller, clock divider, and LED drive logic.

- **Parallel LED Driving**: It uses a Verilog `generate` loop to instantiate a dedicated `PWM` module for every LED in the system.
- **Fading Trail Implementation**: For every time step, it updates a `Level_LED` array where the current active index is set to maximum brightness (Level 5), while all other LEDs gradually decrement their brightness level until they reach zero.
- **Synchronous Integration**: It routes the `divclk` from the clock divider to drive the sequential logic of the controller and the brightness update registers.

3 Internal implementation

3.1 Overall architecture

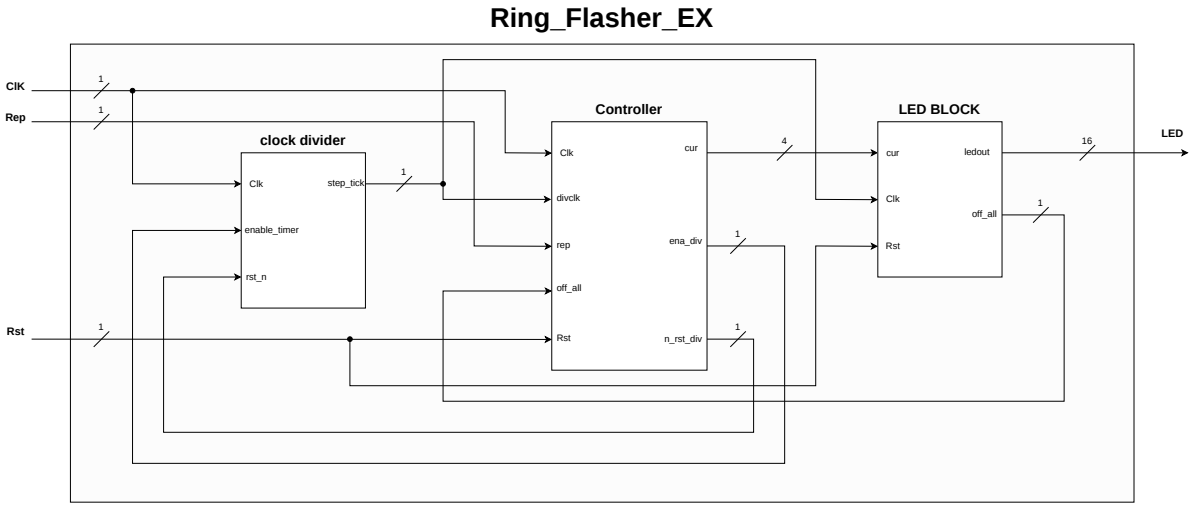

Image 3.1: The Block diagram of the Ring Flasher module.

Table 1: Top-Level Module (Ring_Flasher_EX)

Component	Signal Name	Direction	Width	Description
External Interface	CIK	Input	1-bit	Main system clock source.
	Rep	Input	1-bit	Repeat control signal for sequence logic.
	Rst	Input	1-bit	Global system reset.
	LED	Output	16-bit	Final output bus driving the hardware LEDs.
Clock Divider	step_tick	Output	1-bit	Pulses used to increment sequence timing.
	enable_timer	Input	1-bit	Control signal to activate the internal timer.
	rst_n	Input	1-bit	Active-low reset for the divider sub-module.
Controller	cur	Output	4-bit	Current state/index value for the LED sequence.
	ena_div	Output	1-bit	Enable signal sent to the divider components.
	n_rst_div	Output	1-bit	Dedicated reset for the internal divider logic.
	off_all	Output	1-bit	Global override signal to disable all LED outputs.
LED BLOCK	ledout	Output	16-bit	Internal bus that drives the external LED signals.

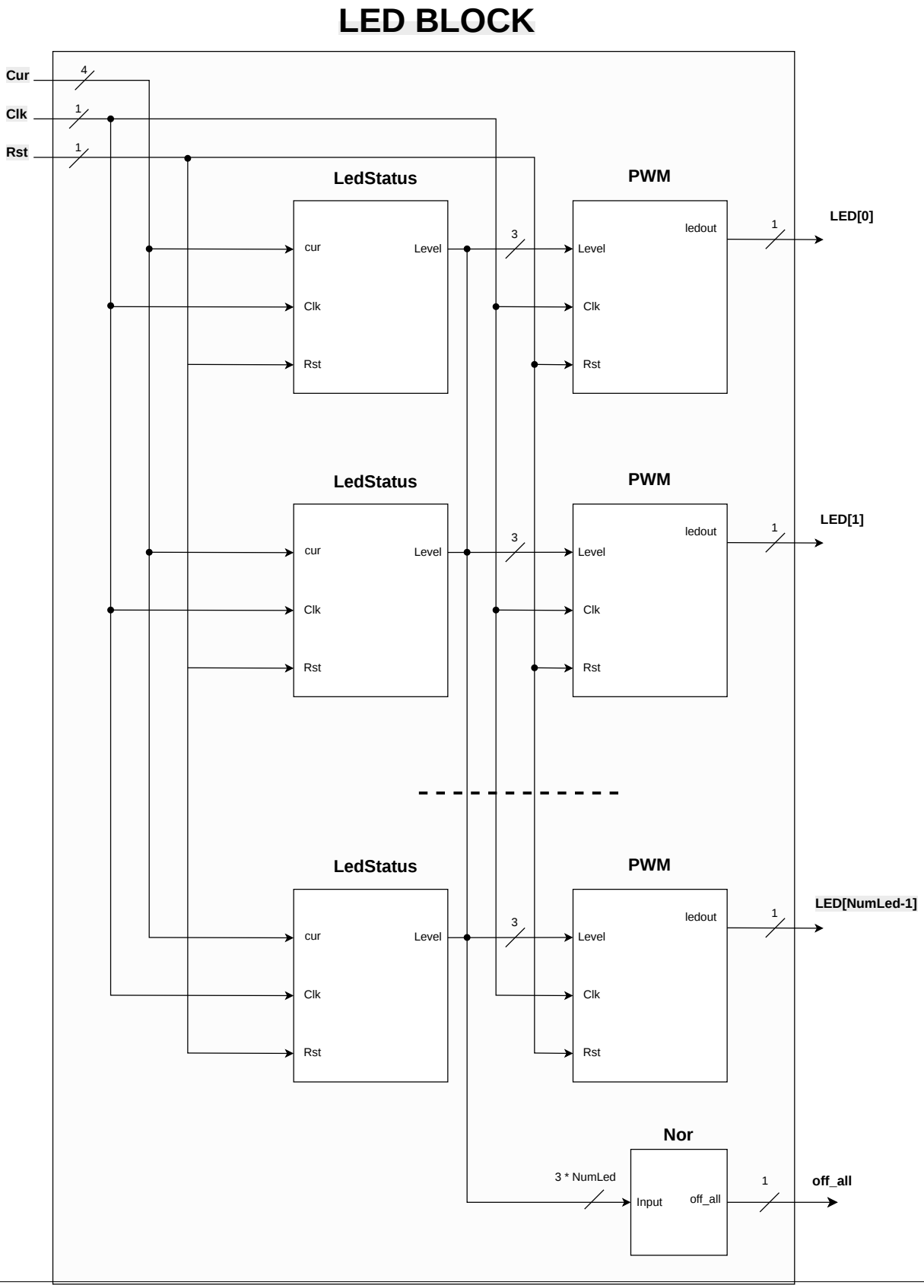

Image 3.2: The Block diagram of the LED BLOCK.

Table 2: LED BLOCK Internal Structure

Sub-Module	Signal Name	Direction	Width	Description
Divider	cur	Input	4-bit	Sequence state input used to determine brightness.
	Level	Output	3-bit	Decoded brightness level for the PWM generator.
PWM	Level	Input	3-bit	Target duty cycle for the pulse width modulation.
	ledout	Output	1-bit	Modulated 1-bit signal for a single LED channel.
Global Signals	Clk	Input	1-bit	Synchronous clock for all internal registers.
	Rst	Input	1-bit	Initialization signal for internal components.

3.2 State machine

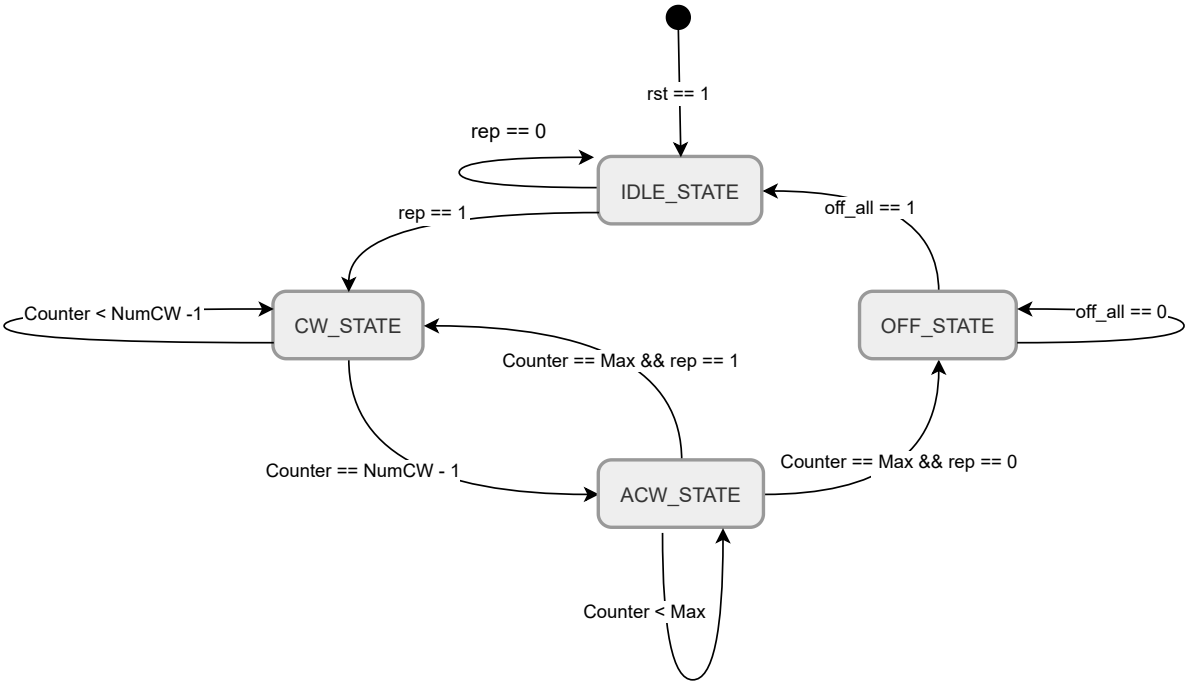

Image 3.3: Finite state machine of the Ring Flasher module.

Parameters The finite state machine is implemented in the `Control` module with the following parameters:

- `NumLed = 16`: number of LEDs in the ring.
- `NumCW = 12`: number of clockwise (CW) steps.
- `NumACW = 8`: number of anti-clockwise (ACW) steps.
- `IDLE_STATE = 0`: idle state.
- `CW_STATE = 1`: clockwise motion state.

- `ACW_STATE = 2`: anti-clockwise motion state.
- `OFF_STATE = 3`: turn-off state until all LEDs fade out.

Variables

- Inputs: `clk`, `divclk`, `rep`, `rst`, `off_all`.
- Outputs: `cur`, `ena_div`, `n_rst_div`.
- State registers: `State`, `NextState`.
- Counters: `Counter`, `NextCounter`.
- LED position registers: `cur_pos`, `next_pos`.

The signal `cur` provides the current LED index to the datapath. The control outputs `ena_div` and `n_rst_div` are used to enable and reset the step-timer clock divider.

State-by-State Behavior

- **IDLE_STATE**: The system waits for `rep = 1`. If `rep = 0`, the divider is disabled and held in reset. When `rep = 1`, the FSM moves to `CW_STATE`, initializes `Counter = 0`, and starts from position `0`.
- **CW_STATE**: At each `divclk` tick, the position moves clockwise and `Counter` is incremented. When `Counter = NumCW - 1`, the FSM transitions to `ACW_STATE`.
- **ACW_STATE**: At each `divclk` tick, the position moves anti-clockwise and `Counter` is incremented. When `Counter = NumCW + NumACW - 1`, the FSM checks `rep`: if `rep = 1`, it returns to `CW_STATE` with `Counter = 0`; otherwise, it enters `OFF_STATE`.
- **OFF_STATE**: No new LED position is injected (position is forced to an invalid index). The datapath keeps decreasing LED intensity until all LEDs are off. When `off_all = 1`, the FSM returns to `IDLE_STATE`.

4 Simulation

4.1 Testbench

1. Simulation Configuration and Parameters

The testbench configures the Device Under Test (DUT) with specific hardware and timing constants to match the intended operational environment:

- **System Clock**: The simulation is set to a frequency of 5 kHz. This corresponds to a clock period of 200,000 ns.
- **Hardware Parameters**: The system is instantiated with 16 LEDs, 12 clockwise steps (`NumCW`), and 8 anti-clockwise steps (`NumACW`).
- **Temporal Resolution**: A `TimeStep` parameter of 500 ms is applied to define the duration of each LED state transition.
- **Cycle Definitions**: The testbench calculates `RUN20_CYCLES` and `RUN5_CYCLES` based on the clock frequency to simulate specific time intervals.

2. Clock Generation and Initialization

- **Clock Stimulus**: A perpetual clock signal is generated by toggling the `clk` register at half-period intervals (100,000 ns).
- **System Reset**: Upon initialization, the `reset` signal is driven high and held for 10 clock cycles to ensure all internal registers reach a known state.
- **Initial Conditions**: The repeat signal (`rep`) is initially set to 0 to prevent the FSM from advancing before the reset is released.

3. Test Stimulus Phases

The simulation executes two distinct phases to verify the controller's reactivity to user input:

- **Phase 1 (Active Operation)**: The `rep` signal is set to 1. The simulation runs for the duration of `RUN20_CYCLES` (representing approximately 40 seconds of system time) to observe the continuous repetition of the clockwise and anti-clockwise LED sequence.

- **Phase 2 (Sequence Termination)**: The `rep` signal is cleared to 0. The simulation continues for `RUN5_CYCLES` (representing 5 seconds) to verify that the system completes its current cycle and transitions to the `OFF_STATE` correctly before the simulation finishes.

4. Output Monitoring

The testbench utilizes the `$monitor` task to provide real-time logging of the simulation time, the 16-bit `LedOut` bus, the `rep` status, and the `reset` state. This ensures that any deviations in the output pattern can be identified relative to the control signals.

4.2 Simulation results

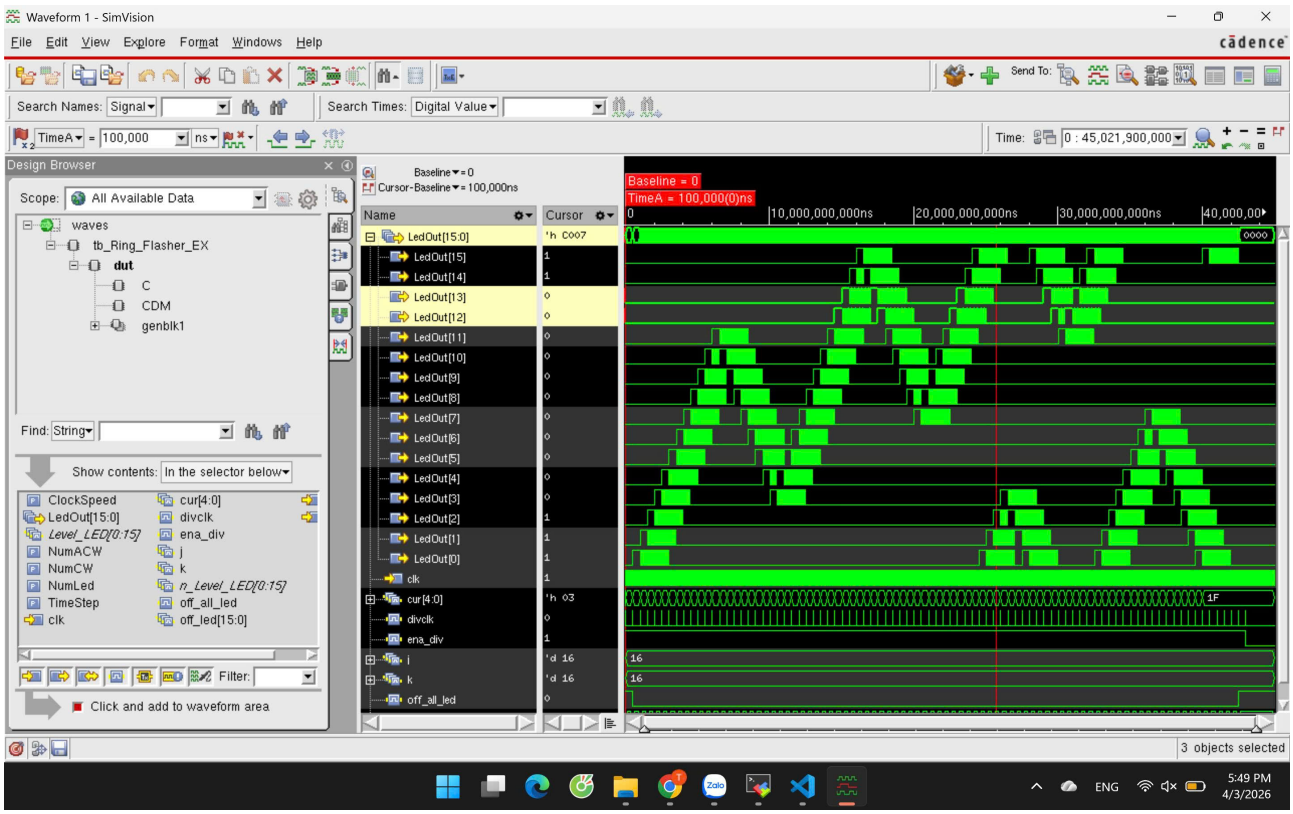

Image 4.1: The simulation result of the Ring Flasher module.

The simulation results shown in the waveform verify that the `Ring_Flasher_EX` design operates correctly according to the following specifications:

- **Sequence and Direction Accuracy**: The waveform clearly demonstrates the state transitions from `CW_STATE` to `ACW_STATE`. The active bit in the `LedOut[15:0]` bus increments for exactly 12 steps

(Clockwise) before reversing direction for 8 steps (Anti-clockwise) as defined by the `NumCW` and `NumACW` parameters.

- **Timing and Clock Synchronization**: The `divclk` pulses occur at precise intervals derived from the `TimeStep` parameter of 500ms. The simulation time axis confirms that the LED transitions are synchronized with these pulses, ensuring the system maintains the required 500ms-per-step pace.
- **Fading Trail Implementation**: The multi-level brightness requirement is met through the PWM logic. As seen in the waveform, the `LedOut` bits exhibit varying duty cycles; the current position is at 100% (Level 5), while the preceding bits show a decreasing duty cycle (Level 4, 3, 2, 1), confirming the mathematical decrement of the brightness level.
- **Control and Repeat Logic**: The simulation validates the response to the `rep` signal. When `rep` is high, the FSM successfully loops through the CW and ACW sequences. Upon setting `rep` to 0, the system correctly completes its current cycle and transitions to the `OFF_STATE`, darkening all LEDs once the fading process is finalized.
- **Reset Integrity**: At the start of the simulation, the `reset` signal successfully initializes all registers to 0, ensuring no unintended LED activity occurs until the controller leaves the `IDLE_STATE`.

Conclusion: Based on the visual evidence from the waveform and the recorded logs, the design successfully meets all functional and timing requirements specified for the Ring Flasher system.